Day1-Intro

Day1

React js

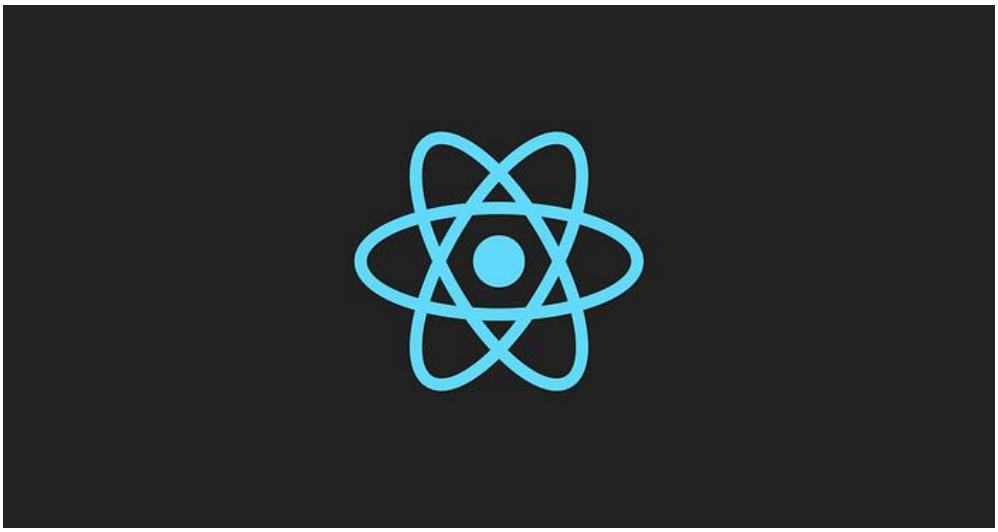
React is an open-source **JavaScript library** developed by Meta Platforms for building **declarative, component-based user interfaces (UIs)**, particularly for single-page applications (SPAs) and interactive web applications.

ReactJS: A brief history

React is a popular free and open-source front end Javascript library that is used for building UI components and user interfaces.

It was created by Facebook and is still maintained by them along with an ever growing community of companies and individual developers.

React is commonly used as the code base for SPA's (Single Page Applications).



But how did React get its start?

In 2011, as Facebook's Ad app began to pick up major momentum, their code began seeing a new updates and team members which became overwhelming and hard to manage after a while. In an attempt to help remedy this problem, Jordan Walke created FaxJS, a prototype which made this process more efficient. This was the birth of what would later evolve into the version of React that we all currently know and love today.

In 2012, Facebook bought a little company called Instagram, who had just 13 employees at the time, for $1 Billion. After its acquisition, Instagram wanted to adopt much of Facebook's new technology. This put pressure on the parent company to decouple React from Facebook and make it open-source, which is exactly what was done. In late May 2013, Jordan Walke finally introduced what was now called 'React' and open-sourced the material.

Some notable features which have helped make React so beloved by developers are its use of Components, the Virtual DOM, Lifecycle Methods, JSX, and React hooks.

React applications

Many popular websites and web applications are built with React due to its flexibility, performance, and scalability. Here are some well-known examples:

1. Facebook

- React was originally developed by Facebook, and it's used extensively across the platform. Both the Facebook website and the Instagram app use React in their user interfaces.

2. Instagram

- Instagram's web app is built using React, including features like the Explore page, photo/video uploads, and direct messaging.

3. Netflix

- Netflix uses React on its client-side for fast and smooth performance. The front-end architecture of their platform makes use of React for its user interface rendering.

4. Airbnb

- Airbnb uses React for its rich, interactive user experience. It powers many of their features, from search and filters to booking management.

5. WhatsApp Web

- The web version of WhatsApp, which offers messaging services directly from a browser, is built using React.

6. Uber

- Uber's web dashboard for managing rides and the admin panel for business management are built using React, providing a responsive and fluid experience.

7. Dropbox

- Dropbox uses React to manage its complex user interface and provide a seamless file-sharing experience.

8. BBC

- The British Broadcasting Corporation (BBC) uses React on its website to deliver content-rich pages with interactive elements and smooth navigation.

9. New York Times

- The New York Times uses React for interactive articles, multimedia content, and its crossword puzzle feature.

10. Atlassian Products (Jira, Confluence, Bitbucket)

- Atlassian’s suite of products, including Jira, Confluence, and Bitbucket, use React to deliver rich, interactive user experiences for project management and collaboration.

11. GitHub

- GitHub's new UI, especially the code editor and notifications page, makes use of React to enhance performance and usability.

12. Reddit

- Reddit's redesign was done using React. The modern, responsive interface you see today is powered by React on both desktop and mobile versions.

13. Shopify

- Shopify uses React for its admin interface, providing a dynamic and interactive experience for shop owners managing their stores.

14. Trello

- Trello uses React for its web app, offering a smooth drag-and-drop interface for task and project management.

15. CodeSandbox

- CodeSandbox, an online code editor for web applications, is built using React. It allows developers to write, test, and share React code directly in the browser.

16. Discord

- Discord's web app uses React for rendering its user interface, allowing for real-time updates and seamless user interactions.

17. Salesforce

- Salesforce’s Lightning Design System, used for building enterprise applications, is built with React.

18. Khan Academy

- Khan Academy uses React to create interactive and educational content, enabling a dynamic and responsive learning experience.

19. Pinterest

- Pinterest’s web interface is built using React, ensuring fast and smooth interactions, especially when browsing through a large number of images.

20. Microsoft Products (Outlook, Teams)

- Microsoft has used React for its web versions of Outlook and Teams, providing a responsive and interactive user experience.

React: Component-Based Architecture, Virtual DOM, and SPA — Notes

1. Component-Based Architecture

Definition:
React applications are built using independent, reusable pieces called **components**. Each component encapsulates its own UI logic and presentation.

Why React Uses It:

- Promotes code reusability.
- Simplifies maintenance.
- Enables modular development.
- Improves scalability for large applications.
- Allows teams to work on different UI parts independently.

Example:

App

├── Header

├── Sidebar

├── UserProfile

├── Dashboard

└── Footer

Each component can be developed, tested, and reused separately.

Benefits:

- Reusable code
- Easier debugging
- Better maintainability
- Faster development
- Separation of concerns

---

2. Virtual DOM

Definition:
The Virtual DOM is a lightweight JavaScript representation of the real browser DOM.

Why React Uses It:
Directly updating the browser DOM is expensive and can slow applications.

React:

1. Creates a Virtual DOM copy.
2. Updates the Virtual DOM when state changes.
3. Compares the new Virtual DOM with the previous one (**Diffing**).
4. Updates only the changed elements in the real DOM (**Reconciliation**).

Process:

```
State Change
      ↓
Virtual DOM Updated
      ↓
Diffing Algorithm
      ↓
Find Changes
      ↓
Update Real DOM Efficiently
```

Benefits:

- Faster UI updates
- Better performance
- Reduced DOM manipulation
- Improved user experience

Example:

Before:

```
<h1>Hello</h1>
```

After state change:

```
<h1>Hello John</h1>
```

React updates only the text node instead of re-rendering the entire page.

---

3. Single Page Application (SPA)

Definition:
A Single Page Application loads a single HTML page and dynamically updates content without full page reloads.

Traditional Multi-Page Application (MPA):

```
User Clicks Link
      ↓
Request Sent
      ↓
Server Generates New Page
      ↓
Entire Page Reloads
```

React SPA:

```
User Clicks Link
      ↓
React Router Changes View
      ↓
Only Required Components Update
      ↓
No Full Page Reload
```

Why React Is Popular for SPAs:

- Fast navigation
- Smooth user experience
- Reduced server load
- Efficient client-side rendering
- Better responsiveness

Examples:

- Gmail
- Facebook
- Twitter/X
- Netflix

React Project Setup

We use:

- Node.js
- Vite
- React

Step 1 – Open VS Code

Open terminal.

Step 2 – Create React Project

```
npm create vite@latest
```

Step 3 – Enter Project Name

```
doctorsapp
```

Step 4 – Select Framework

Choose:

```
React
```

Step 5 – Select Variant

Choose:

```
JavaScript
```

Step 6 – Move into Project

```
cd doctors-app
```

Step 7 – Install Packages

```
npm install
```

Step 8 – Run Project

```
npm run dev
```

Output

```
http://localhost:5173
```

Open in browser.

Detailed breakdown of the **React + Vite project folder structure**

📁 React + Vite Project Folder Structure Explained

When you create a project with:

npm create vite@latest my-app -- --template react

You get this structure:

```
my-app/
├── node_modules/
├── public/
├── src/
│   ├── App.css
│   ├── App.jsx
│   ├── index.css
│   └── main.jsx
├── .gitignore
├── index.html
├── package.json
├── vite.config.js
└── README.md
```

📂 Folder-by-Folder & File-by-File Explanation

🏗️ Root Level Files & Folders

📁 node_modules/

- Stores all installed npm packages
- Auto-created after npm install
- **Don't edit this folder!**

📄 package.json

- Contains project metadata
- Lists all dependencies, scripts, and project name
- Example:

```
{
  "name": "my-app",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0"
  },
  ...
}
```

📄 vite.config.js

- Vite’s configuration file
- You can add plugins, aliases, or environment settings here
- Basic config:

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
});
```

📄 index.html

- The main HTML file used in your app (not in /public)
- It contains the root where React renders
- Also links the entry script:

```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
```

☑ **Vite directly uses this file**, unlike CRA.

📄 .gitignore

- Tells Git which files/folders to ignore
- Common ones:
- node_modules/
- dist/
- .env

📄 README.md

- Documentation file for your project
- Great for giving an overview to other devs

📁 public/

- Stores **static assets** (images, logos, etc.)
- Files here are served **as-is**
- Accessed via URL paths like /logo.png
- **Avoid placing sensitive files here**

📁 src/

Where all your **React code** lives. Let's break it down:

📄 main.jsx

- **Entry point** of your React app
- Renders <App /> into the root element

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

📄 App.jsx

- The **root component** of your app
- Acts like a layout or router wrapper

```
function App() {
  return <h1>Hello Vite + React</h1>;
}

export default App;
```

📄 App.css and index.css

- Optional CSS files
- App.css for styling specific to the App component
- index.css for global styles (like resets, fonts, etc.)

☑ You can replace with Tailwind, CSS Modules, or styled-components if needed.

🗂️ You can add more folders to src/:

Folder	Purpose
components/	Reusable components (like Button, Card)
pages/	Page-level components (Home, About)
assets/	Images, fonts, icons
hooks/	Custom React hooks
context/	Context providers (auth, theme)
services/	API functions or data fetching

!

JSX (JavaScript XML)

🔹 What is JSX?

JSX stands for **JavaScript XML**. It is a **syntax extension** for JavaScript, commonly used with React to describe what the UI should look like. It looks like HTML but has the full power of JavaScript.

Why Use JSX?

- Makes code **easier to write and understand**.
- Looks like **HTML**, which is familiar to web developers.
- Allows embedding **JavaScript expressions** within the markup.
- JSX gets **compiled to React.createElement() calls** under the hood.

🔧 Babel

🔹 What is Babel?

Babel is a **JavaScript compiler** that transforms modern JavaScript and JSX into code that browsers can understand.

🔧 JSX & Babel:

JSX is **not valid JavaScript**, so browsers can't read it directly. **Babel** converts JSX into plain JavaScript using the React.createElement() function.

💡 Example:

Input (JSX):

```
const greeting = <h1>Hello, JSX!</h1>;
```

Babel Output:

```
const greeting = React.createElement('h1', null, 'Hello, JSX!');
```

So, Babel + JSX = Modern and browser-friendly code.

📙 JSX Rules in React

JSX has **some strict rules and best practices**. Here's a full breakdown:

1. Return Only One Parent Element

A JSX expression must have **one root element**.

❌ Invalid:

```
const Parent = () => {
   return (
      <h1>Hello</h1>
      <p>World</p>
    );
};
```

☑ Valid:

```
const Parent = () => {
   return (
      <div>
        <h1>Hello</h1>
        <p>World</p>
      </div>
    );
};
```

---

2. Use className Instead of class

class is a reserved keyword in JavaScript.

❌ Invalid:

```
  <div class="box"></div>
```

☑ Valid:

```
  <div className="box"></div>
```

---

3. Use camelCase for Attributes

HTML attributes are written in **camelCase** in JSX.

☑ Examples:

```
<input type="text" />
<button onClick={handleClick}>Click</button>
```

---

4. Close All Tags

All elements must be **self-closed** if they don't have children.

❌ Invalid:

```
<img src="logo.png">
```

☑ Valid:

```
<img src="logo.png" />
```

---

5. JSX Allows JavaScript Expressions Inside {}

You can embed any valid JS expression using curly braces {}.

```
  const name = "React";
  return (
 <div>
  <h1>Hello, {name}</h1>
 </div>
);
```

---

6. No if Statements Inside JSX

You can't use if inside JSX directly – instead use:

- **Ternary operator** condition ? true : false
- **Logical AND** condition && expression

☑ Examples:

```
{isLoggedIn ? <h1>Welcome</h1> : <h1>Please log in</h1>}

{showMessage && <p>This is visible only when showMessage is true</p>}
```

---

7. Fragments (<>...</>) to Avoid Extra Nodes

React components must return one root element, but you can use **fragments** if you don't want to add extra HTML elements.

```
<>
  <h1>Title</h1>
  <p>Description</p>
</>
```

---

8. Inline Styling in JSX

Use style={{}} with camelCase properties.

```
<div style={{ backgroundColor: 'skyblue', padding: '10px' }}>Hello</div>
```

---

9. Comments in JSX

Use curly braces with regular JS comments:

```
  {/* This is a comment */}
```

10. Component name should start with capital letter

📘 React Components

🔹 What is a Component?

- A **component** is a reusable bit of code responsible for rendering **part of the user interface (UI)**.
- It can be **as small as a button** or **as large as an entire page**.

🔸 Types of Components

1. **Functional Components**
    - These are JavaScript functions that return JSX.

- Simple and commonly used in modern React.

2. **Class Components**
    - Use ES6 classes.
    - Useful when working with older codebases or needing state/lifecycle methods (though now mostly replaced by hooks).

React Code- Day 1

Component Based Architecture

A React application is divided into small reusable pieces called Components.

Project Structure

```
doctorapp
│
├── node_modules
├── public
│
├── src
│   │
│   ├── assets
│   │
│   ├── components
│   │   ├── Navbar.jsx
│   │   ├── Section.jsx
│   │   └── styles.css
│   │
```

```
│   ├── App.jsx
│   ├── App.css
│   ├── index.css
│   └── main.jsx
│
├── package.json
└── vite.config.js
```

Components Created

Navbar Component

File

`src/components/Navbar.jsx`

Code

```
function Navbar() {
  return (
    <header>
      <h1>Doctor app</h1>

      <div>
        <button>Home</button>
        <button>Add doctor</button>
      </div>
    </header>
  );
}
```

```
export default Navbar;
```

Section Component

File

src/components/Section.jsx

Code

```
function Section() {
  return (
    <section className="s1">
      <div>
        <h1>Mr. Doctor app</h1>
        <p>Find your doctors for your problems</p>
      </div>

      <div>
        <img
          src="https://img.magnific.com/free-photo/beautiful-house-with-nature-elements_23-2151848749.jpg?t=st=1780014392~exp=1780017992~hmac=782b6affb0beb13b8285941d8db80fcbdfd5decb01564b7fe4fcc878a146b417"
          alt=""
        />
      </div>
    </section>
  );
}
```

```
export default Section;
```

Root Component

File

src/App.jsx

Code

```
import Navbar from "./components/Navbar";
import Section from "./components/Section";
import "./components/styles.css";

function App() {
  return (
    <div>
      <Navbar />
      <Section />
    </div>
  );
}

export default App;
```

Styling

styles.css

Universal Selector

```
*{
    margin:0;
    padding:0;
    box-sizing:border-box;
}
```

Purpose

Removes default browser spacing.

Navbar Styling

```
header{
    display:flex;
    justify-content:space-between;
    align-items:center;
    background:blueviolet;
    padding:20px 30px;
    color:white;
    font-weight:900;
    font-size:2rem;
}

```

Button Styling

```
button{
    padding:10px;
```

```
    margin:5px;
}

```

Section Styling

```
.s1{
    display:flex;
    justify-content:space-around;
    align-items:center;
    background:blueviolet;
    color:white;
    font-size:3rem;
}

```

Child Div Styling

```
.s1 > div{
    flex-basis:50%;
    padding:30px;
}
```

flex-basis

Determines how much space an item occupies inside a flex container.

```
50% = Half Width

```

Application Flow

main.jsx

↓

App.jsx

↓

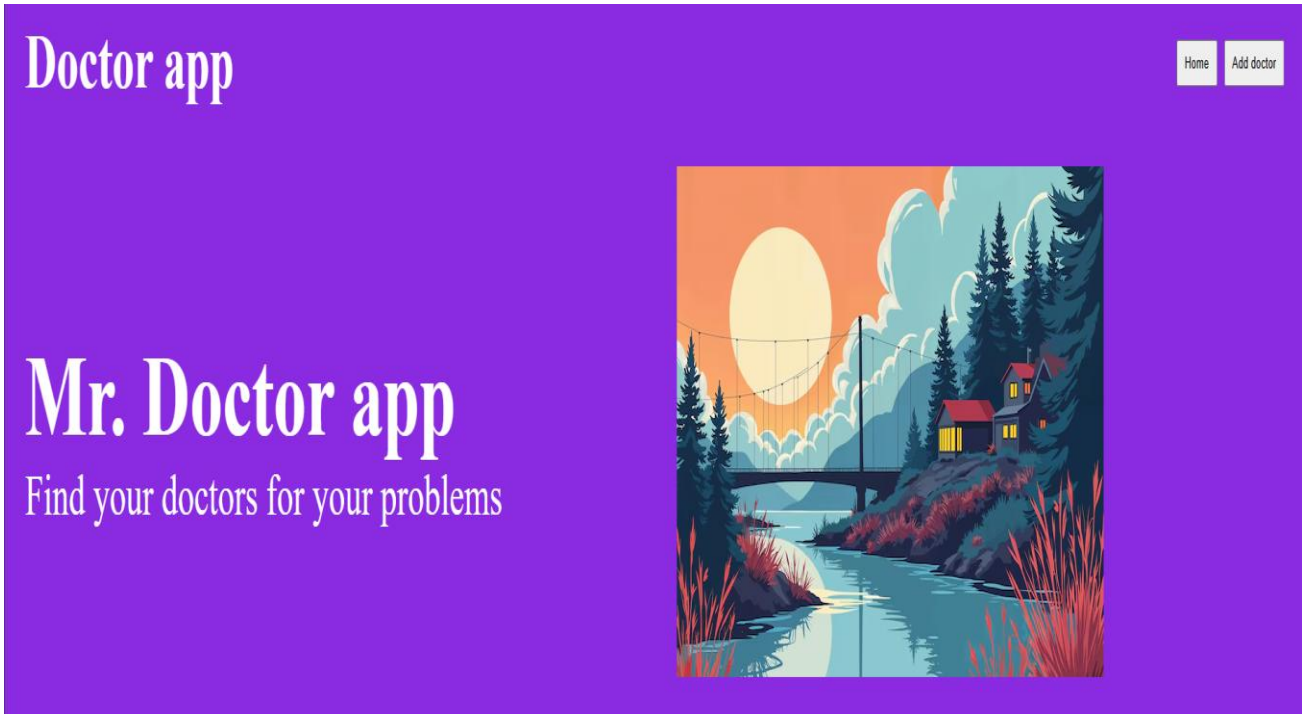
Navbar.jsx

↓

Section.jsx

React starts rendering from main.jsx.

Output



day2-props

React Day 2 Notes

Project Folder Structure

```
src
|
├── components
|   ├── Navbar.jsx
|   ├── Section.jsx
|   ├── Doctorcard.jsx
|   └── styles.css
|
├── App.jsx
└── main.jsx
```

React Code- Day 2

Part 1: Creating Multiple Cards Using Same Component + Flexbox

Concept

A component can be reused multiple times to display the same UI structure.

Doctorcard.jsx

```
function Doctorcard() {
  return (
    <div className="card">
```

```
      <div>
        <img
          src="https://cdn-icons-
png.flaticon.com/512/387/387561.png"
          width="100"
        />
      </div>

      <h2>JOHN</h2>
      <p>Muscles Specialist</p>
      <p>Male</p>

      <button>View Details</button>
    </div>
  );
}

export default Doctorcard;
```

App.jsx

```
import Doctorcard from "./components/Doctorcard";
import Navbar from "./components/Navbar";
import Section from "./components/Section";
import "./components/styles.css";

function App() {
  return (
    <div>
```

```
      <Navbar />
      <Section />

      <div className="doctorparent">
        <Doctorcard />
        <Doctorcard />
        <Doctorcard />
        <Doctorcard />
        <Doctorcard />
        <Doctorcard />
        <Doctorcard />
      </div>
    </div>
  );
}

export default App;
```

---

styles.css

```
.card {
  box-shadow: 0px 0px 2px 2px black;
  padding: 10px;
  text-align: center;
}

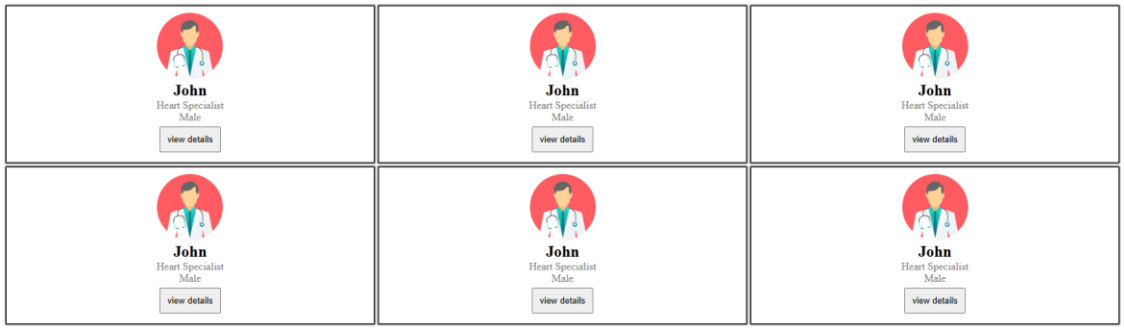
.card > p {
  font-weight: 300;
```

```
  color: grey;
}

.doctorparent {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
  justify-content: center;
  margin: 20px;
}

.doctorparent > div {
  flex-basis: 30%;
}
```

Part-1 Output



Part 2: Dynamic Cards Using Props

Technical Definition of Props

Props (Properties) are read-only inputs passed from a parent component to a child component for transferring dynamic data.

App.jsx

```
import Doctorcard from "./components/Doctorcard";
import Navbar from "./components/Navbar";
import Section from "./components/Section";
import "./components/styles.css";

function App() {
  return (
    <div>
      <Navbar />
      <Section />

      <div className="doctorparent">
        <Doctorcard
          name="Sivaram"
          specialization="Bones"
          gender="male"
        />

        <Doctorcard
          name="Teja"
          specialization="Muscles"
          gender="male"
        />

        <Doctorcard
          name="Vasavi"
          specialization="Psycho"
```

```
          gender="female"
        />

        <Doctorcard
          name="Madhu"
          specialization="Orthocare"
          gender="male"
        />

        <Doctorcard
          name="Aditya"
          specialization="Skin Specialist"
          gender="male"
        />

        <Doctorcard
          name="John"
          specialization="Lungs"
          gender="male"
        />
      </div>
    </div>
  );
}

export default App;
```

Receiving Props Using props Object

Doctorcard.jsx

```
function Doctorcard(props) {
  return (
    <div className="card">
      <div>
        <img
          src="https://cdn-icons-
png.flaticon.com/512/387/387561.png"
          width="100"
        />
      </div>

      <h2>{props.name}</h2>
      <p>{props.specialization}</p>
      <p>{props.gender}</p>

      <button>View Details</button>
    </div>
  );
}

export default Doctorcard;
```

Destructuring Props

Technical Definition

Destructuring is a JavaScript feature used to extract values from objects or arrays and store them directly in variables.

Doctorcard.jsx

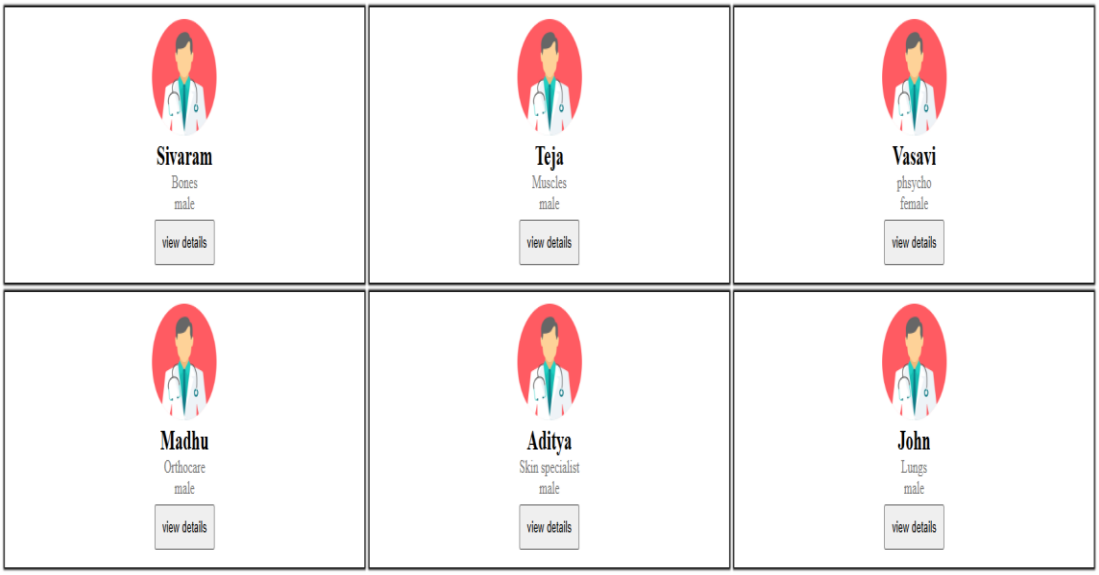
```
function Doctorcard({ name, specialization, gender }) {
  return (
    <div className="card">
      <div>
        <img
          src="https://cdn-icons-
png.flaticon.com/512/387/387561.png"
          width="100"
        />
      </div>

      <h2>{name}</h2>
      <p>{specialization}</p>
      <p>{gender}</p>

      <button>View Details</button>
    </div>
  );
}

export default Doctorcard;
```

Part 2 Output



Summary

Part 1

- Created reusable DoctorCard component
- Called same component multiple times
- Used Flexbox for layout
- Learned component reusability

Part 2

- Passed dynamic data using Props
- Received data using `props`
- Used Props Destructuring
- Built dynamic and reusable cards

day3-useState and forms

React Day 3 - Forms and useState

Day 3 Overview

Topics Covered

- [x] Creating Forms in React
- [x] Form Styling
- [x] Controlled Components
- [x] useState Hook
- [x] Handling Input Fields
- [x] Handling Select Dropdown
- [x] onChange Event
- [x] Form Submission
- [x] preventDefault()
- [x] Creating Objects from Form Data

Folder Structure

```
src
|
├── components
|   ├── Addnewdoctor.jsx
|   └── styles.css
|
├── App.jsx
└── main.jsx
```

Part 1: Creating Doctor Form UI

Technical Definition

A **Form** is a collection of input fields used to collect user data.

Addnewdoctor.jsx

```
function Addnewdoctor() {
  return (
    <div>
      <h1>Add New Doctor</h1>

      <form className="form-container">
        <input
          type="text"
          placeholder="Enter Doctor Name"
          className="text-field"
        />

        <input
          type="text"
          placeholder="Enter Age"
          className="text-field"
        />

        <select>
          <option>Select Gender</option>
```

```
          <option value="Male">Male</option>
          <option value="Female">Female</option>
        </select>

        <input
          type="text"
          placeholder="Enter Specialization"
          className="text-field"
        />

        <input
          type="text"
          placeholder="Enter Salary"
          className="text-field"
        />

        <button>Add Doctor</button>
      </form>
    </div>
  );
}

export default Addnewdoctor;
```

Styling

styles.css

```
.form-container {
```

```
  width: 33%;
  margin: auto;
  border: 2px solid black;
  padding: 20px;
}

.text-field,
select,
.form-container button {
  display: block;
  padding: 10px;
  width: 100%;
  margin: 10px;
}

.form-container button {
  background: black;
  color: white;
}
```

Part 2: Using useState for Form Inputs

Technical Definition of useState

`useState` is a React Hook used to create and manage state inside functional components.

Syntax

```
const [state, setState] = useState(initialValue);
```

Creating State Variables

```
const [name, setName] = useState("");
const [gender, setGender] = useState("");
const [age, setAge] = useState("");
const [salary, setSalary] = useState("");
const [specialization, setSpecialization] = useState("");
```

Technical Definition of Controlled Component

A **Controlled Component** is a form element whose value is controlled by React State.

Controlled Inputs

```
<input
  value={name}
  onChange={(e) => setName(e.target.value)}
  type="text"
/>

<input
  value={age}
  onChange={(e) => setAge(e.target.value)}
  type="text"
/>

<select
  value={gender}
```

```
  onChange={(e) => setGender(e.target.value)}
>
  <option>Select Gender</option>
  <option value="Male">Male</option>
  <option value="Female">Female</option>
</select>

<input
  value={specialization}
  onChange={(e) => setSpecialization(e.target.value)}
  type="text"
/>

<input
  value={salary}
  onChange={(e) => setSalary(e.target.value)}
  type="text"
/>
```

Part 3: Form Submission

Technical Definition of onSubmit

`onSubmit` is an event that executes when a form is submitted.

Technical Definition of preventDefault()

`preventDefault()` prevents the browser's default form submission behavior and page refresh.

Submit Function

```
function handlesubmit(e) {
  e.preventDefault();

  let newdoctor = {
    name,
    age,
    gender,
    salary,
    specialization,
  };

  console.log(newdoctor);
}
```

Connecting Form with Submit Function

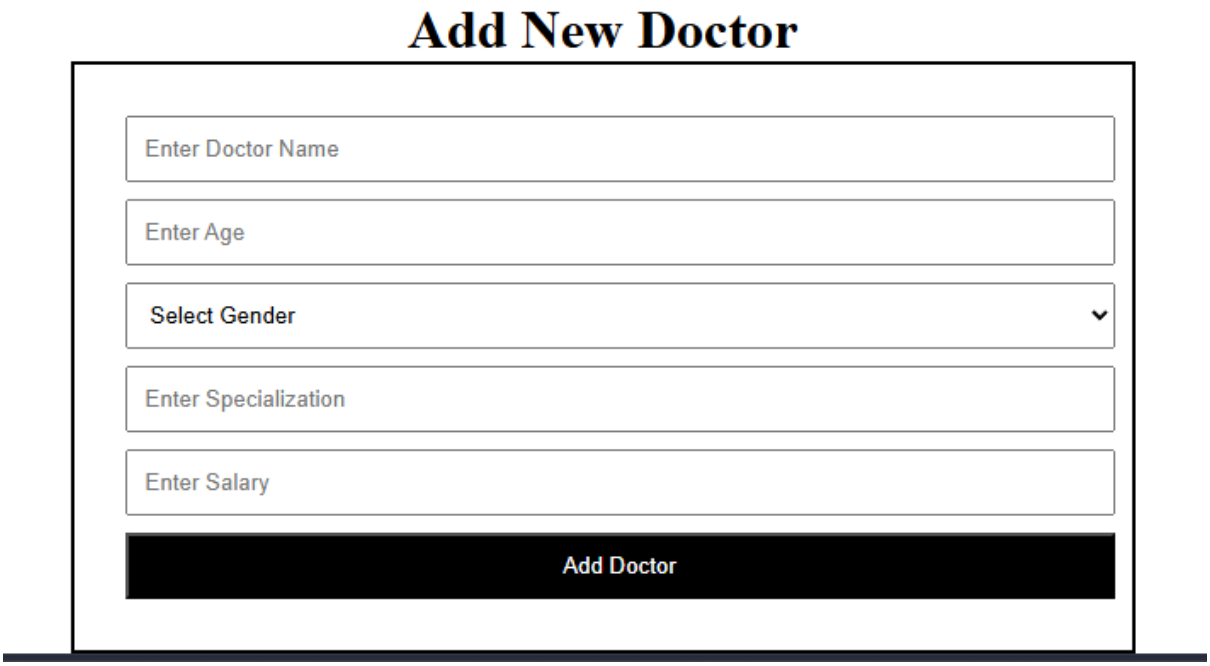
```
<form
  className="form-container"
  onSubmit={handlesubmit}
>
```

Submit Button

```
<button type="submit">
  Add Doctor
```

```
</button>
```

OUTPUT



Day 3 Summary

Concepts Learned

- Forms in React
- Form Styling
- useState Hook
- State Variables
- Controlled Components
- onChange Event
- Handling Input Fields
- Handling Dropdowns
- onSubmit Event
- preventDefault()
- Creating Objects
- Form Data Collection

day4-Map,key,ternary

React Day 4 - useEffect, Map, Key, Conditional Rendering & Passing Data from Form

Topics Covered

- ☑ useEffect Hook
- ☑ Fetching Data
- ☑ Dependency Array
- ☑ map()
- ☑ key Prop
- ☑ Conditional Rendering
- ☑ Passing Data from Parent to Child
- ☑ Updating State Arrays
- ☑ Spread Operator
- ☑ Dynamic Rendering

Folder Structure

```
src
│
├── components
│   ├── Addnewdoctor.jsx
│   ├── Home.jsx
│   ├── Doctorcard.jsx
│   └── styles.css
```

```
│
├── App.jsx
└── main.jsx
```

Part 1 - Home.jsx

Concepts Covered

useEffect

Technical Definition:

useEffect is a React Hook used to perform side effects inside functional components.

Dependency Array

```
useEffect(() => {


}, []);
```

Runs only once when the component loads.

map()

Technical Definition:

map() is a JavaScript array method used to iterate over arrays and return UI elements dynamically.

key

Technical Definition:

A key is a unique identifier used by React to efficiently update list items.

Conditional Rendering

Technical Definition:

Displaying different UI elements based on a condition.

Home.jsx

```
import { useState, useEffect } from "react";
import Doctorcard from "./Doctorcard";

function Home() {
  let [doctors, setDoctors] = useState([]);

  function fetchdata() {
    let data = [
      {
        id: 1,
        name: "Teja",
        age: 26,
        gender: "Male",
        specialization: "Muscles",
        salary: 7000000,
      },

      {
        id: 2,
        name: "Sam",
        age: 26,
        gender: "Male",
        specialization: "Bones",
        salary: 4000000,
      },

      {
```

```
      id: 3,
      name: "Anu",
      age: 25,
      gender: "Female",
      specialization: "Heart",
      salary: 5000000,
    },
  ];

  setDoctors(data);
}

useEffect(() => {
  fetchdata();
}, []);

return (
  <div>
    {doctors.length > 0 ? (
      <div className="doctorparent">
        {doctors.map((doctor) => {
          return (
            <Doctorcard
              key={doctor.id}
              name={doctor.name}
              gender={doctor.gender}
              specialization={doctor.specialization}
            />
```

```
            );
          })}
        </div>
      ) : (
        <h1>No Doctors Found</h1>
      )}
    </div>
  );
}

export default Home;
```

Part 2 - Addnewdoctor.jsx

Concepts Covered

Form Submission

Collecting user data through input fields.

preventDefault()

Stops page refresh when form is submitted.

```
e.preventDefault();
```

Creating Object

```
let formdata = {
  name,
  age,
  gender,
  salary,
```

```
  specialization,
  id: Date.now(),
};
```

Passing Props

Sending data from parent component to child component.

```
<Home newdoctor={newdoctor}/>
```

Addnewdoctor.jsx

```
import { useState } from "react";
import Home from "./Home";

function Addnewdoctor() {
  const [name, setName] = useState("");
  const [gender, setGender] = useState("");
  const [age, setAge] = useState("");
  const [salary, setSalary] = useState("");
  const [specialization, setSpecialization] = useState("");

  let [newdoctor, setNewdoctor] = useState(null);

  function handlesubmit(e) {
    e.preventDefault();

    let formdata = {
      name,
      age,
```

```
    gender,
    salary,
    specialization,
    id: Date.now(),
  };

  setNewdoctor(formdata);
}

return (
  <div>
    <h1 style={{ textAlign: "center" }}>Add New Doctor</h1>

    <form
      action=""
      className="form-container"
      onSubmit={handlesubmit}
    >
      <input
        value={name}
        onChange={(e) => setName(e.target.value)}
        type="text"
        placeholder="Enter Doctor Name"
        className="text-field"
      />

      <input
        value={age}
```

```
  onChange={(e) => setAge(e.target.value)}
  type="text"
  placeholder="Enter Age"
  className="text-field"
/>

<select
  value={gender}
  onChange={(e) => setGender(e.target.value)}
>
  <option>Select Gender</option>
  <option value="Male">Male</option>
  <option value="Female">Female</option>
</select>

<input
  value={specialization}
  onChange={(e) => setSpecialization(e.target.value)}
  type="text"
  placeholder="Enter Specialization"
  className="text-field"
/>

<input
  value={salary}
  type="text"
  onChange={(e) => setSalary(e.target.value)}
  placeholder="Enter Salary"
```

```
          className="text-field"
        />

        <button type="submit">Add Doctor</button>
      </form>

      <Home newdoctor={newdoctor} />
    </div>
  );
}

export default Addnewdoctor;
```

Part 3 - Home.jsx (Receiving New Doctor Data)

Additional Concepts Covered

Props

Receiving data from parent component.

`function Home({ newdoctor })`

useEffect with Dependency

```
useEffect(() => {

}, [newdoctor]);
```

Runs whenever newdoctor changes.

Updating Array State

```
setDoctors((prev) => [...prev, newdoctor]);
```

Previous State

`prev` represents the current doctors array.

Spread Operator (...)

Copies all existing array elements.

```
[...prev, newdoctor]
```

Dynamic Rendering

When a new doctor is submitted, React automatically updates the UI.

Home.jsx

```
import { useState, useEffect } from "react";
import Doctorcard from "./Doctorcard";

function Home({ newdoctor }) {
  let [doctors, setDoctors] = useState([]);

  function fetchdata() {
    let data = [
      {
        id: 1,
        name: "Teja",
        age: 26,
        gender: "Male",
        specialization: "Muscles",
        salary: 7000000,
      },
```

```
    {
      id: 2,
      name: "Sam",
      age: 26,
      gender: "Male",
      specialization: "Bones",
      salary: 4000000,
    },

    {
      id: 3,
      name: "Anu",
      age: 25,
      gender: "Female",
      specialization: "Heart",
      salary: 5000000,
    },
  ];

  setDoctors(data);
}

useEffect(() => {
  fetchdata();
}, []);

useEffect(() => {
```

```
    if (newdoctor) {
      setDoctors((prev) => [...prev, newdoctor]);
    }
  }, [newdoctor]);

  return (
    <div>
      {doctors.length > 0 ? (
        <div className="doctorparent">
          {doctors.map((doctor) => {
            return (
              <Doctorcard
                key={doctor.id}
                name={doctor.name}
                gender={doctor.gender}
                specialization={doctor.specialization}
              />
            );
          })}
        </div>
      ) : (
        <h1>No Doctors Found</h1>
      )}
    </div>
  );
}

export default Home;
```

day5-Search Filter and Deployment

React Day 5 - Search and Filter Functionality

Day 5 Overview

On Day 5, we enhanced the existing Doctor Management Application by adding:

☑ Search Input

☑ Specialization Dropdown Filter

☑ Real-Time Filtering

☑ Controlled Input Field

☑ Controlled Select Dropdown

The doctor list updates automatically based on the search text and selected specialization.

Topics Covered

- Search Functionality
- Specialization Filtering
- Input Field Handling
- Select Dropdown Handling
- Controlled Components
- useState
- filter()
- Dynamic Rendering

Home.jsx

New Concepts Added

Search State

Stores the text entered by the user.

```
let [search, setSearch] = useState("");
```

Specialization State

Stores the selected specialization.

```
let [specialization, setSpecialization] = useState("");
```

Search Input

Used to search doctors by name.

```
<input
  type="text"
  className="text-field"
  value={search}
  placeholder="search doctor"
  onChange={(e) => setSearch(e.target.value)}
/>
```

What We Learned

- Controlled Input
- onChange Event
- State Updates
- Real-Time Search

Select Dropdown

Used to filter doctors by specialization.

```
<select
  className="text-field"
  value={specialization}
  onChange={(e) => setSpecialization(e.target.value)}
>
  <option value="Muscles">Muscles</option>
  <option value="Bones">Bones</option>
  <option value="Heart">Heart</option>
</select>
```

What We Learned

- Controlled Select
- Dropdown Handling
- State Management
- Filtering Data

Filtering Doctors

```
const filtereddoctors = doctors.filter((val) => {
  return (
    val.name.toLowerCase().includes(search.toLowerCase()) &&
    (
      specialization == "" ||
      val.specialization == specialization
    )
  );
});
```

What We Learned

- filter()
- Search Logic
- Multiple Conditions
- Real-Time Filtering

Home.jsx Code

```
import { useState, useEffect } from "react";
import Doctorcard from "./Doctorcard";

function Home({ newdoctor }) {
  let [doctors, setDoctors] = useState([]);
  let [search, setSearch] = useState("");
  let [specialization, setSpecialization] = useState("");

  function fetchdata() {
    let data = [
      {
        id: 1,
        name: "Teja",
        age: 26,
        gender: "Male",
        specialization: "Muscles",
        salary: 7000000,
      },
      {
        id: 2,
        name: "Sam",
        age: 26,
        gender: "Male",
        specialization: "Bones",
        salary: 4000000,
```

```
    },
    {
      id: 3,
      name: "Anu",
      age: 25,
      gender: "Female",
      specialization: "Heart",
      salary: 5000000,
    },
  ];

  setDoctors(data);
}

useEffect(() => {
  fetchdata();
}, []);

useEffect(() => {
  if (newdoctor) {
    setDoctors((prev) => [...prev, newdoctor]);
  }
}, [newdoctor]);

const filtereddoctors = doctors.filter((val) => {
  return (
    val.name.toLowerCase().includes(search.toLowerCase()) &&
    (
```

```
      specialization == "" ||
      val.specialization == specialization
    )
  );
});

return (
  <div>
    <div className="filters">
      <input
        type="text"
        className="text-field"
        value={search}
        placeholder="search doctor"
        onChange={(e) => setSearch(e.target.value)}
      />

      <select
        className="text-field"
        value={specialization}
        onChange={(e) => setSpecialization(e.target.value)}
      >
        <option value="">Select Specialization</option>
        <option value="Muscles">Muscles</option>
        <option value="Bones">Bones</option>
        <option value="Heart">Heart</option>
      </select>
    </div>
```

```
      {filtereddoctors.length > 0 ? (
        <div className="doctorparent">
          {filtereddoctors.map((doctor) => {
            return (
              <Doctorcard
                key={doctor.id}
                name={doctor.name}
                gender={doctor.gender}
                specialization={doctor.specialization}
              />
            );
          })}
        </div>
      ) : (
        <h1>No Doctors Found</h1>
      )}
    </div>
  );
}

export default Home;
```

GitHub Deployment

Step 1 – Initialize Git

```
git init
```

□

Step 2 – Add Files

```
git add .
```

□

Step 3 – Commit

```
git commit -m "Doctors App"
```

□

Step 4 – Create Repository

Create:

- doctors-app repository

Step 5 – Connect Repository

```
git remote add origin YOUR_GITHUB_LINK
```

□

Step 6 – Push Code

```
git push -u origin main
```

□

---

6. Vercel Deployment

Go to:
Vercel

Steps

1. Login using GitHub
2. Click:
    - Add New Project
3. Import:

    - doctors-app repository
4. Click:
    - Deploy

Output

Students get:

- Live application link

Example:

```
https://doctors-app.vercel.app
```

What Students Learned Today

Students completed:

- Search functionality
- Filter functionality
- filter()
- map()
- Conditional rendering
- GitHub deployment
- Vercel deployment

And improved:

```
Doctors App
```

day6-Router Dom

Day 6 – Routing (Doctors App)

Topics Covered

1. React Router Setup
2. Routes
3. useNavigate
4. useParams

What is Routing?

Routing means:

- Switching between pages

Examples:

- Home Page
- About Page
- Contact Page

In React:

- Page does NOT reload completely

Only component changes.

Why Routing?

Suppose user clicks:

`View Details`

We should navigate to:

`/doctor/2`

and display:

- Complete doctor details

For this we use:

`React Router`

---

1. Install React Router

Open terminal:

```
npm install react-router-dom
```

---

2. Setup BrowserRouter
main.jsx

```
import React from 'react'
import ReactDOM from 'react-dom/client'

import { BrowserRouter } from "react-router-dom"

import App from './App'

import './index.css'

ReactDOM.createRoot(document.getElementById('root')).render(

  <BrowserRouter>

    <App />

  </BrowserRouter>

)
```

What is BrowserRouter?

BrowserRouter enables:

- Routing in React application

Without BrowserRouter:

- Routes will not work

---

3. Create Routes
App.jsx

```
import Navbar from "./components/Navbar"
import Section from "./components/Section"
import Home from "./pages/Home"
```

```
import AddDoctor from "./pages/AddDoctor"

import {
  Routes,
  Route
} from "react-router-dom"

function App() {

  return (

    <div>

      <Navbar />
       <Section/>
      <Routes>

        <Route
          path="/"
          element={<Home />}
        />

        <Route
          path="/add-doctor"
          element={<AddDoctor />}
        />

      </Routes>

    </div>
  )
}

export default App
```

Understanding Routes

Route	Page
/	Home Page
/add-doctor	Add Doctor Page

---

4. Navbar Navigation

Currently buttons do nothing.

Now we will navigate pages.

---

useNavigate Hook

useNavigate is used for:

- Page navigation

Import useNavigate
Navbar.jsx

```
import { useNavigate } from "react-router-dom"

```

Create navigate Variable

```
const navigate = useNavigate()

```

Update Navbar.jsx

```
import { useNavigate } from "react-router-dom"

function Navbar() {

  const navigate = useNavigate()

  return (

    <div className="navbar">

      <h1>Doctors App</h1>

      <div>

        <button onClick={() => navigate("/")}>
          Home
        </button>
```

```
        <button
          onClick={() => navigate("/add-doctor")}
        >
          Add Doctor
        </button>

      </div>

    </div>
  )
}

export default Navbar
```

Understanding navigate()

`navigate("/")`

Moves user to:

- Home Page

Output

Students can now:

- Switch pages dynamically

Without:

- Page reload

---

5. Create Doctor Details Page

Now we will create:

- Separate doctor details page

When user clicks:

`View Details`

Navigate to:

```
/doctor/2
```

Create New Page
pages/DoctorDetails.jsx

```
function DoctorDetails() {

  return (

    <div className="container">

      <h1>Doctor Details Page</h1>

    </div>
  )
}

export default DoctorDetails
```

Add Route
App.jsx

```
import DoctorDetails from "./pages/DoctorDetails"
```

Add Route

```
<Route
  path="/doctor/:id"
  element={<DoctorDetails />}
/>
```

Understanding :id

```
/doctor/:id
```

Means:

- Dynamic route parameter

Examples:

```
/doctor/1
/doctor/2
/doctor/10
```

---

6. useNavigate in DoctorCard

Now when user clicks:

- View Details

Navigate to:

- Doctor Details Page

Update DoctorCard.jsx

```
import { useNavigate } from "react-router-dom"

function DoctorCard({
  id,
  name,
  age,
  gender,
  specialization,
  salary
}) {

  const navigate = useNavigate()

  return (

    <div className="card">

      <img
        src="https://cdn-icons-png.flaticon.com/512/387/387561.png"
        width="100"
      />

      <h2>{name}</h2>

      <p>{specialization}</p>

      <p>Age: {age}</p>
```

```
      <p>Gender: {gender}</p>

      <p>Salary: {salary}</p>

      <button
        onClick={() => navigate(`/doctor/${id}`)}
      >
        View Details
      </button>

    </div>
  )
}

export default DoctorCard
```

Important Understanding

```
navigate(`/doctor/${id}`)
```

Example:

```
/doctor/2
```

Pass id from Home.jsx

```
<DoctorCard
  key={doctor.id}
  id={doctor.id}
  name={doctor.name}
  age={doctor.age}
  gender={doctor.gender}
  specialization={doctor.specialization}
  salary={doctor.salary}
/>
```

---

7. useParams Hook

Now we need:

- Route ID

Example:

- `/doctor/2`

- Need:

- `2`

- For this we use:

- `useParams`

Import useParams
DoctorDetails.jsx

```
import { useParams } from "react-router-dom"
```

Get Route Parameter

```
const { id } = useParams()
```

Understanding

If URL is:

- `/doctor/5`

- Then:

- `id = 5`

Display ID
DoctorDetails.jsx

```
import { useParams } from "react-router-dom"
import { useEffect,useState } from "react"

function Doctordetails() {
    let {id}=useParams()
    let [doctor,setDoctor]=useState([])

    async function fetchdata(){
```

```
            let data = [
                {
                    id: 1,
                    name: "Teja",
                    age: 26,
                    gender: "Male",
                    specialization: "Muscles",
                    salary: 7000000
                },


                {
                    id: 2,
                    name: "Sam",
                    age: 26,
                    gender: "Male",
                    specialization: "Bones",
                    salary: 4000000
                },


                {
                    id: 3,
                    name: "Anu",
                    age: 25,
                    gender: "Female",
                    specialization: "Heart",
                    salary: 5000000
                }
            ]



  let doc=data.find((val)=> val.id==id)
  setDoctor(doc)

  }
  useEffect(()=>{
      fetchdata()
  },[])
return (
  <div>
      <h1>Doctor details</h1>
      <h1>{doctor.name}</h1>
      <h2>{doctor.gender}</h2>
      <h6>{doctor.age}</h6>
      <h2>{doctor.specialization}</h2>
      <h2>{doctor.salary}</h2>
```

```
    </div>
  )
}

export default Doctordetails
```

Output

If URL:

```
/doctor/2
```

Output:

```
Doctor ID: 2
```

Complete App.jsx

```
import Navbar from "./components/Navbar"
import Section from "./components/Section"

import Home from "./pages/Home"
import AddDoctor from "./pages/AddDoctor"
import DoctorDetails from "./pages/DoctorDetails"

import {
  Routes,
  Route
} from "react-router-dom"

function App() {

  return (

    <div>

      <Navbar />
      <Section />

      <Routes>

        <Route
          path="/"
```

```
          element={<Home />}
        />

        <Route
          path="/add-doctor"
          element={<AddDoctor />}
        />

        <Route
          path="/doctor/:id"
          element={<DoctorDetails />}
        />

      </Routes>

    </div>
  )
}

export default App
```

Folder Structure

```
src
│
├── components
│     ├── Navbar.jsx
│     └── DoctorCard.jsx
│
├── pages
│     ├── Home.jsx
│     ├── AddDoctor.jsx
│     └── DoctorDetails.jsx
│
├── App.jsx
├── App.css
```

---

day7-Api Get

Day 7– APIs and GET Requests (Doctors App)

Topics Covered

1. REST API Introduction
2. HTTP Methods
3. Axios Setup
4. GET Request
5. Handling Response
6. map() with API Data
7. Props Usage

---

1. What is REST API?

REST API helps:

- Frontend communicate with backend

Real-Time Understanding

Frontend:

- React Application

Backend:

- Server + Database

REST API acts like:

- Middleman

Example Flow

```
React App
    ↓
API Request
    ↓
Backend Server
    ↓
```

```
Database
    ↓
Data Response
```

API We Are Using

```
https://doctorapibackend.onrender.com/doctors
```

Sample API Data


```
{
  "name":"Teja",
  "salary":"7000000",
  "gender":"Male",
  "age":"26",
  "specialization":"Muscles",
  "id":2
}
```


---

2. HTTP Methods

HTTP Methods are used to:

- Perform operations on server

Important Methods

Method	Purpose
GET	Fetch data
POST	Add data
PUT	Update data
DELETE	Delete data

Today Focus

Today we will use:

```
GET
```

GET Method

GET is used to:

- Fetch data from server

Example:

- Get all doctors

---

3. Axios Setup
What is Axios?

Axios is a library used for:

- API requests

Why Axios?

- Easy syntax
- Better readability
- Mostly used in React projects

Install Axios

Open terminal:

```
npm install axios
```

Import Axios
Home.jsx

```
import axios from "axios"
```

---

4. GET Request

Now we will:

- Fetch doctors from API

Create Doctors State
Home.jsx

```
const [doctors, setDoctors] = useState([])
```

Why Array?

Because API returns:

- Multiple doctors

Create API Function

```
const getDoctors = async () => {

  let response = await axios.get(
    "https://doctorapibackend.onrender.com/doctors"
  )

  console.log(response)

}
```

Understanding async/await

API calls take time.

So we use:

- async
- await

Understanding axios.get()

```
axios.get()
```

Used for:

- GET request

Understanding Response

`response`

Contains:

- API response object

Actual Data

`response.data`

Contains:

- Doctors array

Update State

`setDoctors(response.data)`

Complete Function

```
const getDoctors = async () => {

  let response = await axios.get(
    "https://doctorapibackend.onrender.com/doctors"
  )

  console.log(response.data)

  setDoctors(response.data)
}
```

---

5. useEffect with API

Now we want:

- Fetch doctors automatically after page load

For this we use:

- useEffect

Import useEffect

```
import { useEffect } from "react"
```

---

useEffect Example

```
useEffect(() => {

  getDoctors()

}, [])
```

Understanding []

```
[]
```

Means:

- Run only once
- After component load

Real-Time Flow

```
Home Page Opens
      ↓
useEffect Runs
      ↓
API Request Sent
      ↓
Data Received
      ↓
Doctors Display
```

---

6. map() with API Data

Now doctors state contains:

- API doctors data

We will display using:

- map()

Display Doctors

```
{
  doctors.map((doctor) => (

    <DoctorCard
      key={doctor.id}
      id={doctor.id}
      name={doctor.name}
      age={doctor.age}
      gender={doctor.gender}
      specialization={doctor.specialization}
      salary={doctor.salary}
    />

  ))
}
```

Why key?

React needs:

- Unique identification

Best practice:

- Use database id

---

7. Props Usage

Now we pass API data:

- Parent → Child

Parent Component
Home.jsx

```
<DoctorCard
  name={doctor.name}
  specialization={doctor.specialization}
/>
```

Child Component
DoctorCard.jsx

```
function DoctorCard({
  name,
  specialization
}) {

  return (

    <div>

      <h2>{name}</h2>

      <p>{specialization}</p>

    </div>
  )
}
```

Complete Home.jsx

```
import { useEffect, useState } from "react"

import axios from "axios"

import DoctorCard from "../components/DoctorCard"

function Home() {
```

```
  const [doctors, setDoctors] = useState([])

  const getDoctors = async () => {

    let response = await axios.get(
      "https://doctorapibackend.onrender.com/doctors"
    )

    console.log(response.data)

    setDoctors(response.data)
  }

  useEffect(() => {

    getDoctors()

  }, [])

  return (

    <div className="container">

      <h1>All Doctors</h1>
{doctors.length>0?? (
      <div className="doctors-container">

        {
          doctors.map((doctor) => (

            <DoctorCard
              key={doctor.id}
              id={doctor.id}
              name={doctor.name}
              age={doctor.age}
              gender={doctor.gender}
              specialization={doctor.specialization}
              salary={doctor.salary}
            />

          ))
        }

      </div>): <h1>No doctors found</h1>
}
    </div>
  )
}
```

```
export default Home
```

DoctorCard.jsx

```
import { useNavigate } from "react-router-dom"

function DoctorCard({
  id,
  name,
  age,
  gender,
  specialization,
  salary
}) {

  const navigate = useNavigate()

  return (

    <div className="card">

      <img
        src="https://cdn-icons-png.flaticon.com/512/387/387561.png"
        width="100"
      />

      <h2>{name}</h2>

      <p>{specialization}</p>

      <p>Age: {age}</p>

      <p>Gender: {gender}</p>

      <p>Salary: {salary}</p>

      <button
        onClick={() => navigate(`/doctor/${id}`)}
      >
        View Details
      </button>

    </div>
  )
```

```
}

export default DoctorCard
```

Basic Styling
App.css

```
.container{
  padding:20px;
}

.doctors-container{
  display:flex;
  gap:20px;
  flex-wrap:wrap;
}

.card{
  width:250px;
  border:1px solid gray;
  padding:20px;
  background:white;
}
```

Folder Structure

```
src
│
├── components
│     ├── Navbar.jsx
│     └── DoctorCard.jsx
│
├── pages
│     ├── Home.jsx
│     ├── AddDoctor.jsx
│     └── DoctorDetails.jsx
│
├── App.jsx
├── App.css
```

What Students Learned Today

Students completed:

- REST API understanding
- HTTP methods
- Axios setup
- GET request
- API response handling
- map() with API data
- Props usage

And connected:

```
React Frontend + Backend API

```

day8=Post ,Upd and Del

Day 8 – POST, PUT, DELETE Operations (Doctors App)

Topics Covered

1. POST Request
2. PUT Request
3. DELETE Request
4. Form Submission to API
5. Updating UI Dynamically

Revision – HTTP Methods

Method	Purpose
GET	Fetch data
POST	Add data
PUT	Update data
DELETE	Delete data

Today Focus

Today we will perform:

- CRUD Operations

What is CRUD?

Operation	Meaning
Create	Add data

Read	Fetch data
Update	Edit data
Delete	Remove data

API Endpoint

```
https://doctorapibackend.onrender.com/doctors
```

---

1. POST Request
What is POST?

POST is used to:

- Add new data to server

Example:

- Add new doctor

Real-Time Flow

```
User Fills Form
      ↓
Submit Form
      ↓
POST Request
      ↓
Data Stored in Backend
```

AddDoctor.jsx
Import Axios

```
import axios from "axios"
```

Create Form States

```
const [name, setName] = useState("")
const [age, setAge] = useState("")
const [gender, setGender] = useState("")
const [specialization, setSpecialization] = useState("")
const [salary, setSalary] = useState("")
```

Create handleSubmit Function

```
const handleSubmit = async (e) => {

  e.preventDefault()

  const newDoctor = {
    name,
    age,
    gender,
    specialization,
    salary
  }

  await axios.post(
    "https://doctorapibackend.onrender.com/doctors",
    newDoctor
  )

  alert("Doctor Added Successfully")
}
```

Understanding axios.post()

```
axios.post(url, data)
```

Understanding Parameters

Parameter	Meaning
url	API endpoint
data	Data to send

Connect Form

```
<form onSubmit={handleSubmit}>
```

Complete AddDoctor.jsx

```
import { useState } from "react"

import axios from "axios"

function AddDoctor() {

  const [name, setName] = useState("")
  const [age, setAge] = useState("")
  const [gender, setGender] = useState("")
  const [specialization, setSpecialization] = useState("")
  const [salary, setSalary] = useState("")

  const handleSubmit = async (e) => {

    e.preventDefault()

    const newDoctor = {
      name,
      age,
      gender,
      specialization,
      salary
    }

    await axios.post(
      "https://doctorapibackend.onrender.com/doctors",
      newDoctor
    )

    alert("Doctor Added Successfully")

    setName("")
    setAge("")
    setGender("")
    setSpecialization("")
    setSalary("")
  }
```

```
return (

  <div className="container">

    <h1>Add Doctor</h1>

    <form onSubmit={handleSubmit}>

      <input
        type="text"
        placeholder="Enter Doctor Name"
        value={name}
        onChange={(e)=>setName(e.target.value)}
      />

      <br /><br />

      <input
        type="number"
        placeholder="Enter Age"
        value={age}
        onChange={(e)=>setAge(e.target.value)}
      />

      <br /><br />

      <select
        value={gender}
        onChange={(e)=>setGender(e.target.value)}
      >

        <option value="">
          Select Gender
        </option>

        <option value="Male">
          Male
        </option>

        <option value="Female">
          Female
        </option>

      </select>

      <br /><br />
```

```
        <input
          type="text"
          placeholder="Enter Specialization"
          value={specialization}
          onChange={(e)=>setSpecialization(e.target.value)}
        />

        <br /><br />

        <input
          type="number"
          placeholder="Enter Salary"
          value={salary}
          onChange={(e)=>setSalary(e.target.value)}
        />

        <br /><br />

        <button type="submit">
          Add Doctor
        </button>

      </form>

    </div>
  )
}

export default AddDoctor
```

---

2. DELETE Request
What is DELETE?

DELETE is used to:

- Remove data from server

Example:

- Delete doctor

Real-Time Flow

```
Click Delete
     ↓
```

```
DELETE Request
      ↓
Doctor Removed from Backend
```

Create deleteDoctor Function
Adddoctor.jsx

```
const deleteDoctor = async (id) => {

  await axios.delete(
    `https://doctorapibackend.onrender.com/doctors/${id}`
  )

  getDoctors()
}
```

Understanding URL

```
/doctors/${id}
```

Example:

```
/doctors/2
```

Deletes doctor with ID:

- 2

Pass deleteDoctor Function

Adddoctor.jsx

```
<Home newdoctor={newdoctor} ondel={deletedata} upd={updatedata} />
```

Pass deleteDoctor Function
Home.jsx

```
<DoctorCard
  key={doctor.id}
  id={doctor.id}
```

```
  name={doctor.name}
  age={doctor.age}
  gender={doctor.gender}
  specialization={doctor.specialization}
  salary={doctor.salary}
  deleteDoctor={deleteDoctor}
/>
```

Add Delete Button
DoctorCard.jsx

```
<button
  onClick={() => deleteDoctor(id)}
>
  Delete
</button>
```

Complete Delete Example
DoctorCard.jsx

```
function DoctorCard({
  id,
  name,
  age,
  gender,
  specialization,
  salary,
  deleteDoctor
}) {

  return (

    <div className="card">

      <h2>{name}</h2>

      <p>{specialization}</p>

      <p>Age: {age}</p>

      <p>Gender: {gender}</p>

      <p>Salary: {salary}</p>
```

```
      <button>
        Edit
      </button>

      <button
        onClick={() => deleteDoctor(id)}
      >
        Delete
      </button>

    </div>
  )
}

export default DoctorCard
```

Why getDoctors() Again?

After delete:

- UI should refresh

So we fetch latest data again.

---

3. PUT Request
What is PUT?

PUT is used to:

- Update existing data

Example:

- Update doctor information

Real-Time Flow

```
Click Edit
      ↓
Update Data
      ↓
PUT Request
      ↓
```

```
Backend Updated
```

Simple PUT Example
Adddoctor..jsx

```
const updateDoctor = async (id) => {

  const updatedDoctor = {
    name:"Updated Doctor",
    age:30,
    gender:"Male",
    specialization:"Heart",
    salary:9000000
  }

  await axios.put(
    `https://doctorapibackend.onrender.com/doctors/${id}`,
    updatedDoctor
  )

  getDoctors()
}
```

Understanding axios.put()

```
axios.put(url, updatedData)
```

Pass update Function in Adddoctor.jsx

```
<Home newdoctor={newdoctor} ondel={deletedata} upd={updatedata} />
```

Receive update function in Home.jsx

Pass update Function

```
<DoctorCard
  updateDoctor={updateDoctor}
/>
```

Add Edit Button
DoctorCard.jsx

```
<button
  onClick={() => updateDoctor(id)}
>
  Edit
</button>
```

Updated DoctorCard.jsx

```
function DoctorCard({
  id,
  name,
  age,
  gender,
  specialization,
  salary,
  deleteDoctor,
  updateDoctor
}) {

  return (

    <div className="card">

      <h2>{name}</h2>

      <p>{specialization}</p>

      <p>Age: {age}</p>

      <p>Gender: {gender}</p>

      <p>Salary: {salary}</p>

      <button
        onClick={() => updateDoctor(id)}
      >
        Edit
      </button>

      <button
        onClick={() => deleteDoctor(id)}
      >
        Delete
```

```
      </button>

    </div>
  )
}

export default DoctorCard
```

---

4. Updating UI Dynamically
Problem

After:

- POST
- PUT
- DELETE

UI should refresh automatically.

Solution

Call:

`getDoctors()`

after every operation.

Example

```
await axios.delete(url)

getDoctors()
```

Complete Home.jsx

```
import { useEffect, useState } from "react"

import axios from "axios"

import DoctorCard from "../components/DoctorCard"
```

```
function Home() {

  const [doctors, setDoctors] = useState([])

  const getDoctors = async () => {

    let response = await axios.get(
      "https://doctorapibackend.onrender.com/doctors"
    )

    setDoctors(response.data)
  }

  useEffect(() => {

    getDoctors()

  }, [])

  const deleteDoctor = async (id) => {

    await axios.delete(
      `https://doctorapibackend.onrender.com/doctors/${id}`
    )

    getDoctors()
  }

  const updateDoctor = async (id) => {

    const updatedDoctor = {
      name:"Updated Doctor",
      age:30,
      gender:"Male",
      specialization:"Heart",
      salary:9000000
    }

    await axios.put(
      `https://doctorapibackend.onrender.com/doctors/${id}`,
      updatedDoctor
    )

    getDoctors()
  }

  return (
```

```
    <div className="container">

      <h1>All Doctors</h1>

      <div className="doctors-container">

        {
          doctors.map((doctor) => (

            <DoctorCard
              key={doctor.id}
              id={doctor.id}
              name={doctor.name}
              age={doctor.age}
              gender={doctor.gender}
              specialization={doctor.specialization}
              salary={doctor.salary}
              deleteDoctor={deleteDoctor}
              updateDoctor={updateDoctor}
            />

          ))
        }

      </div>

    </div>
  )
}

export default Home
```

Basic Styling
App.css

```
.container{
  padding:20px;
}

.doctors-container{
  display:flex;
  gap:20px;
  flex-wrap:wrap;
}

.card{
```

```
  width:250px;
  border:1px solid gray;
  padding:20px;
  background:white;
}

button{
  padding:10px 20px;
  margin-right:10px;
  cursor:pointer;
}
```

What Students Learned Today

Students completed:

- POST request
- PUT request
- DELETE request
- Form submission to API
- Dynamic UI updates
- CRUD operations

And connected full CRUD with:

```
Doctors App
```

---

day9-Protected Routes useMemo Custom Hooks

Advanced React Concepts (Doctors App) – Updated Notes

Topics Covered

- Protected Routes
- useMemo
- Custom Hooks
- Error handling

---

1. Protected Routes

What are Protected Routes?

Protected Routes are used to:

☑ Restrict access to pages

☑ Allow only logged-in users

Real-Time Example

User should not access:

- Add Doctor Page

Unless logged in.

Login State

App.jsx

```
const [islogin, setIslogin] = useState(false)
```

Initially:

```
false
```

User clicks login button:

```
<button onClick={() => setIslogin(true)}>
  click to login
</button>
```

Now:

```
islogin = true
```

ProtectedRoute Component

Protectedroute.jsx

```
function Protectedroute({ islogin, children }) {

  if (islogin) {
    return children
  }

  return (
    <div>please login first</div>
  )
}

export default Protectedroute
```

Understanding Children

```
<Protectedroute>

  <Addnewdoctor/>

</Protectedroute>
```

Inside Protectedroute:

□children

□represents:

□<Addnewdoctor/>
□

Using Protected Route

```
□<Route
  path='/add-doctor'
  element={
    <Protectedroute islogin={islogin}>
      <DoctorProvider>
        <Addnewdoctor />
      </DoctorProvider>
    </Protectedroute>
  }
/>
```

□Flow:

```
□islogin ?

YES  → Open Add Doctor Page

NO   → Show "please login first"
```
□

---

2. useMemo Hook

What is useMemo?

useMemo is used for:

☑ Performance Optimization

It prevents unnecessary calculations on every render.

Problem Without useMemo

```
const filtereddoctors = doctors.filter(...)
```

Every render:

- Navbar render
- Search change
- State change

Filter runs again.

Solution

```
const filtereddoctors = useMemo(() => {

  return doctors.filter((val) => {

    console.log("running")

    return (
      val.name
        .toLowerCase()
        .includes(search.toLowerCase())

      &&

      (
        specialization == "" ||
        val.specialization == specialization
      )
    )

  })

}, [search, specialization, doctors])
```

Understanding Dependencies

```
[search, specialization, doctors]
```

Filter runs only when:

- Search changes
- Specialization changes
- Doctors data changes

Otherwise:

Cached value is used.

Filtering Logic

Search Filter

```
val.name
  .toLowerCase()
  .includes(search.toLowerCase())
```

Example:

Search:

```
ra
```

Matches:

```
Ramesh
Rajesh
Pranav
```

Specialization Filter

```
specialization == "" ||
val.specialization == specialization
```

If specialization is empty:

```
Show all doctors
```

Otherwise:

```
Show only selected specialization
```

Benefits

☑ Faster rendering

☑ Better performance

☑ Optimized filtering

---

3. Custom Hooks

What are Custom Hooks?

Custom Hooks are reusable React logic.

Rule:

Hook name must start with:

```
use
```

Examples:

```
useCounter
useDoctors
useFetch
```

Custom Hook Used

useCounter.js

```
import { useState } from "react"

function useCounter() {

  const [count, setCount] = useState(0)

  const increment = () => {
    setCount(count + 1)
  }
```

```
  const decrement = () => {
    setCount(count - 1)
  }

  return {
    count,
    increment,
    decrement
  }
}

export default useCounter
```

Using Custom Hook

App.jsx

```
const {
  count,
  increment,
  decrement
} = useCounter()
```

Display Counter

```
{count}

<button onClick={increment}>
  inc
</button>

<button onClick={decrement}>
  dec
</button>
```

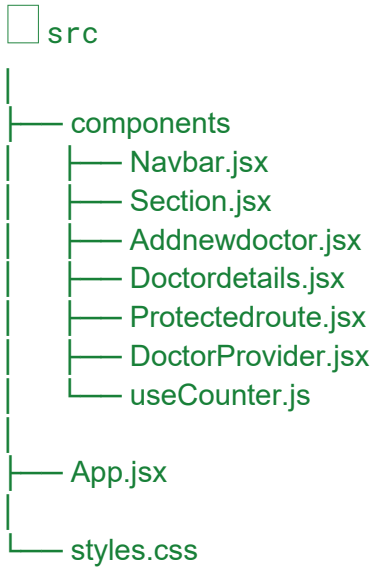
Benefits

☑ Reusable logic

☑ Cleaner code

☑ Better project structure

Folder Structure



day10-usecontext

Day 10 – useContext Hook (Doctors App)

Topics Covered

- createContext
- Context Provider
- useContext Hook
- Sharing Data Between Components
- Avoiding Prop Drilling
- Global State Management
- CRUD Operations Using Context
- useMemo
- Error Handling

What is useContext?

`useContext` allows components to access shared data directly without passing props through multiple components.

Problem Before Context API

Prop Drilling

```
Addnewdoctor

      ↓
    Home
      ↓
 Doctorcard
```

Data and functions were passed as props.

Example:

```
<Home
  newdoctor={newdoctor}
```

```
  ondel={deletedata}
  upd={updatedata}
/>
```

Then Home passes again:

```
<Doctorcard
  ondel={ondel}
  upd={upd}
/>
```

This is called:

Prop Drilling

Solution: Context API

React provides:

createContext()

and

useContext()

to share data globally.

Step 1: Create Context

DoctorProvider.jsx

```
import { createContext } from "react";

export const DoctorContext = createContext();
```

Context is created.

Step 2: Create Provider

```
function DoctorProvider({ children }) {

}
```

Provider wraps components that need access to shared data.

State Stored in Context

```
let [newdoctor, setNewdoctor] = useState(null);
```

Purpose:

```
Track doctor additions
Track doctor updates
Track doctor deletions
```

Delete Doctor Function

```
async function deletedata(id) {

  try {

    await axios.delete(
      `https://doctorapibackend.onrender.com/doctors/${id}`
    )

    alert("doctor deleted")

    setNewdoctor("deleted" + id)

  }

  catch(err){
```

```
    console.log(err)

  }
}
```

Update Doctor Function

```
async function updatedata(id) {

  let data = {

    name: "Manohar",
    age: "57",
    gender: "Male",
    salary: "8900000",
    specialization: "Surgeon",
    id: Date.now()

  }

  try{

    await axios.put(
      `https://doctorapibackend.onrender.com/doctors/${id}`,
      data
    )

    alert("updated")

    setNewdoctor("updated" + id)

  }

  catch(err){

    console.log(err)
```

```
  }

}
```

Provider Value

```
<DoctorContext.Provider

  value={{
    newdoctor,
    setNewdoctor,
    updatedata,
    deletedata
  }}

>

  {children}

</DoctorContext.Provider>
```

Shared globally:

```
newdoctor
setNewdoctor
updatedata
deletedata
```

Complete Provider Code

```
import { createContext, useState } from "react";
import axios from "axios";
```

```
export const DoctorContext = createContext();

function DoctorProvider({ children }) {

 let [newdoctor, setNewdoctor] = useState(null);

 async function deletedata(id) {

   try {
     await axios.delete(
       `https://doctorapibackend.onrender.com/doctors/${id}`
     );

     alert("doctor deleted");

     setNewdoctor("deleted" + id);

   }

   catch (err) {

     console.log(err);

   }
 }

 async function updatedata(id) {

   let data = {
     name: "Manohar",
     age: "57",
     gender: "Male",
     salary: "8900000",
     specialization: "Surgeon",
     id: Date.now(),
   };

   try {

     await axios.put(
       `https://doctorapibackend.onrender.com/doctors/${id}`,
```

```
        data
      );

      alert("updated");

      setNewdoctor("updaated" + id);

    }

    catch (err) {

      console.log(err);

    }
  }

  return (

    <DoctorContext.Provider

      value={{
        newdoctor,
        setNewdoctor,
        updatedata,
        deletedata
      }}

    >

      {children}

    </DoctorContext.Provider>

  );
}

export default DoctorProvider;
```

Step 3: Wrap Components

App.jsx

```
<DoctorProvider>

  <Addnewdoctor />

</DoctorProvider>
```

All child components can access Context.

---

useContext Hook

Syntax

```
const data = useContext(ContextName)
```

Example:

```
const value =
  useContext(DoctorContext)
```

Add Doctor Component

Import

```
import { useContext } from "react";
import { DoctorContext } from "./DoctorProvider";
```

Access Context

```
let { setNewdoctor } =
  useContext(DoctorContext)
```

Add Doctor

```
async function handlesubmit(e){

  e.preventDefault()

  let formdata = {

    name,
    age,
    gender,
    salary,
    specialization,
    id: Date.now()

  }

  await axios.post(
    "https://doctorapibackend.onrender.com/doctors",
    formdata
  )

  alert("Doctor Added Successfully")

  setNewdoctor(formdata)

}
```

Why setNewdoctor?

When a doctor is added:

`setNewdoctor(formdata)`

Context value changes.

This triggers:

`Home Component`

to re-fetch data.

Home Component

Access Context

```
let { newdoctor } =
  useContext(DoctorContext)
```

Fetch Doctors

```
async function fetchdata(){

  try{

    let apidata =
      await axios.get(
        "https://doctorapibackend.onrender.com/doctors"
      )

    setDoctors(apidata.data)

  }

  catch(err){

    alert("failed")

    console.log(err)

  }

}
```

---

useEffect Dependency

```
useEffect(() => {

  fetchdata()

}, [newdoctor])
```

Flow

When:

```
Doctor Added
Doctor Deleted
Doctor Updated
```

Context changes:

```
newdoctor
```

Then:

```
useEffect runs again
```

Then:

```
fetchdata()
```

Then:

```
Latest doctor list shown
```

DoctorCard Component

Access Context

```
let {
  updatedata,
  deletedata
}
```

```
=
useContext(DoctorContext)
```

Delete Doctor

```
<button
  onClick={() => deletedata(id)}
>
  Del
</button>
```

Update Doctor

```
<button
  onClick={() => updatedata(id)}
>
  Update
</button>
```

Complete Context Flow

```
DoctorProvider
       ↓

Shares

newdoctor
setNewdoctor
updatedata
deletedata

       ↓
```

```
Addnewdoctor
      ↓
useContext
      ↓
setNewdoctor()
      ↓
Context Updates
      ↓
Home Component
      ↓
useEffect Triggered
      ↓
fetchdata()
      ↓
Latest Doctors Displayed
```

Advantages of Context API

- ☑ Avoids Prop Drilling
- ☑ Global State Management
- ☑ Cleaner Components
- ☑ Reusable Functions

- [x] Easy CRUD Operations
- [x] Better Project Structure

Folder Structure

```
src
│
├── components
│       ├── DoctorProvider.jsx
│       ├── Addnewdoctor.jsx
│       ├── Home.jsx
│       ├── Doctorcard.jsx
│
├── App.jsx
```

What Students Learned Today

- [x] createContext()
- [x] Context Provider
- [x] useContext()
- [x] Global State Management
- [x] Sharing Data Between Components
- [x] Avoiding Prop Drilling
- [x] CRUD Operations Through Context
- [x] useEffect with Context
- [x] useMemo Optimization
- [x] Error Handling with try-catch
- [x] Dynamic Updates Across Components Using Context API